

SIMATS ENGINEERING
ASSESSMENT TOOL 2 — ANSWERS

MLA0302 — Reinforcement Learning for Environment and Agent

CO2: Apply Dynamic Programming methods to solve RL problems and derive optimal policies

NAME : SUGUNA G

REG NO : 192425032

Question 1

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. (5 Marks)

State Space

- Current GPS location of each idle driver
- Location and destination of each pending passenger request
- Local demand density (requests per zone) and time of day
- Traffic conditions / estimated travel time between zones

Action Space

- Assign a specific idle driver to a specific passenger request
- Reposition an idle driver toward a high-demand zone
- Hold/delay a match if a better pairing is likely soon

Reward Function

- + reward inversely proportional to passenger waiting time (shorter wait = higher reward)
- + small reward for high driver utilization (fewer idle drivers)
- – penalty for long detours or unfair driver-passenger distance mismatches
- – penalty for unmatched/cancelled requests

Explanation

By rewarding short wait times and penalizing idle/unmatched drivers, the agent learns to proactively position idle drivers near predicted demand hotspots instead of reacting only after a request arrives.

Over many episodes, the learned policy converges toward matches that jointly minimize passenger wait time and maximize driver utilization, which directly improves overall service quality.

Program

```
import random

drivers = ["Driver1", "Driver2", "Driver3"]
passengers = ["Passenger1", "Passenger2", "Passenger3"]

print("Ride Sharing Simulation\n")

for i in range(len(passengers)):

    driver = random.choice(drivers)

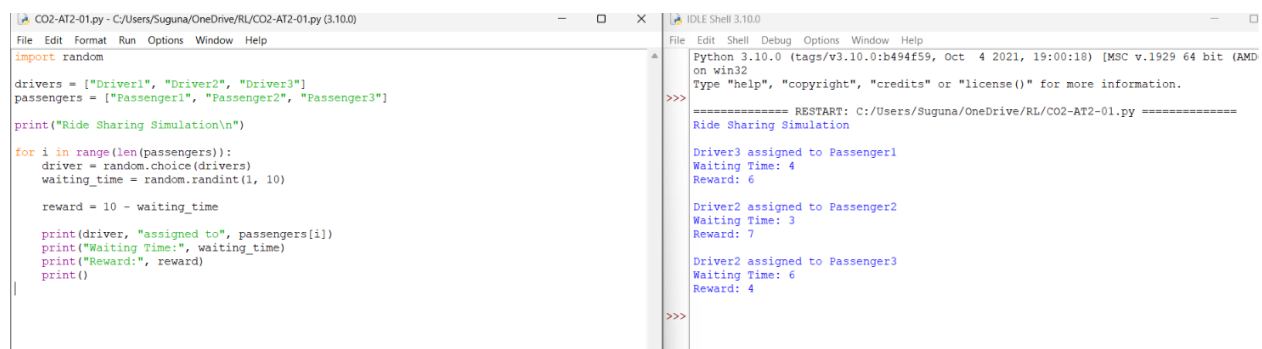
    waiting_time = random.randint(1, 10)

    reward = 10 - waiting_time

    print(driver, "assigned to", passengers[i])
    print("Waiting Time:", waiting_time)
    print("Reward:", reward)

    print()
```

Output



The screenshot shows a Python script in a text editor and its execution output in the IDLE Shell. The script defines a list of drivers and passengers, then iterates over the passengers, randomly assigning a driver to each and calculating a reward based on a random waiting time.

```
File Edit Format Run Options Window Help
import random
drivers = ["Driver1", "Driver2", "Driver3"]
passengers = ["Passenger1", "Passenger2", "Passenger3"]
print("Ride Sharing Simulation\n")
for i in range(len(passengers)):
    driver = random.choice(drivers)
    waiting_time = random.randint(1, 10)
    reward = 10 - waiting_time
    print(driver, "assigned to", passengers[i])
    print("Waiting Time:", waiting_time)
    print("Reward:", reward)
    print()
```

The output in the IDLE Shell shows the execution of the script, displaying the assigned driver, waiting time, and reward for each passenger.

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-01.py =====
Ride Sharing Simulation

Driver3 assigned to Passenger1
Waiting Time: 4
Reward: 6

Driver2 assigned to Passenger2
Waiting Time: 3
Reward: 7

Driver2 assigned to Passenger3
Waiting Time: 6
Reward: 4
>>>
```

2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. (5 Marks)

Exploitation

- Recommend material similar to what the learner has previously engaged with well (high completion rate, high score after study)
- Maximizes short-term relevance using known preferences

Exploration

- Occasionally recommend new topics, formats, or difficulty levels the learner hasn't tried
- Helps discover better-suited content and fills knowledge gaps the learner didn't know to ask for

Explanation

Pure exploitation traps the learner in a narrow loop of familiar content, causing stagnation and missed learning opportunities. Pure exploration wastes the learner's time on irrelevant material and reduces trust in the platform. An epsilon-greedy (or similar) strategy — mostly recommending known-effective material with a small, controlled probability of trying new content — balances reinforcing mastery with expanding the learner's knowledge base over time.

Program

```
import random

# Study Materials

materials = ["Math", "Science", "AI", "Python"]

# Expected rewards for each material

rewards = {

    "Math": 6,

    "Science": 7,

    "AI": 8,
```

```
"Python": 10
}

epsilon = 0.2
total_reward = 0

print("E-Learning Recommendation System\n")

for i in range(10):

    # Exploration or Exploitation
    if random.random() < epsilon:

        choice = random.choice(materials)

        strategy = "Exploration"
    else:

        choice = max(rewards, key=rewards.get)

        strategy = "Exploitation"

    reward = rewards[choice]

    total_reward += reward

    print("Recommendation", i + 1)

    print("Strategy :", strategy)

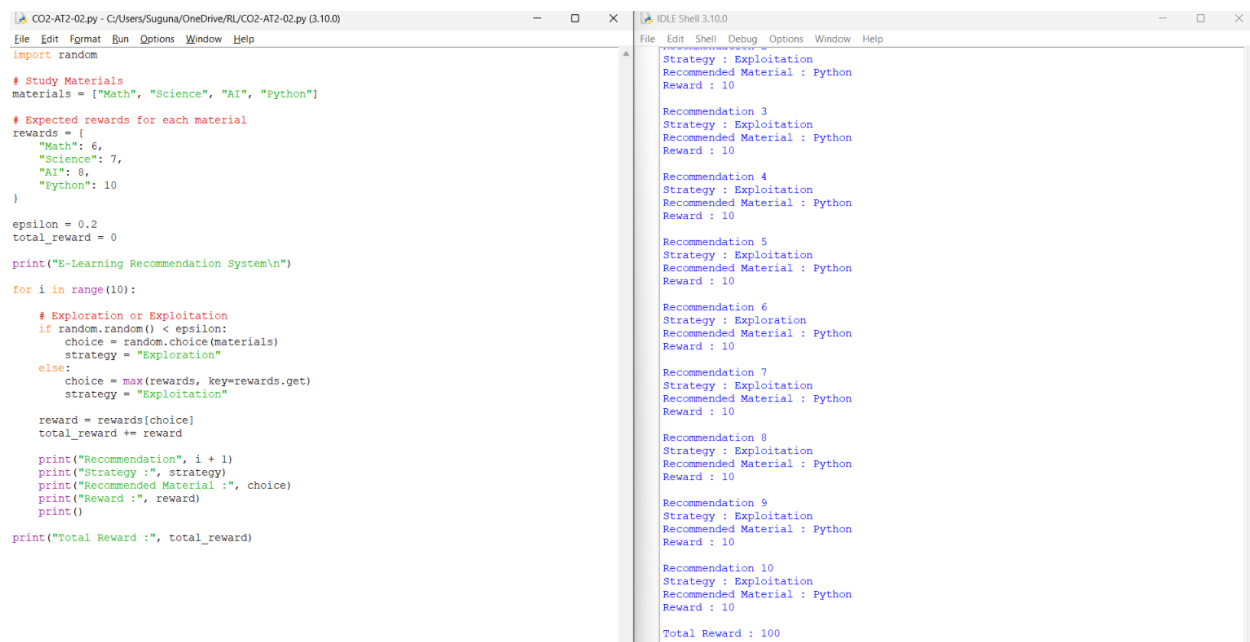
    print("Recommended Material :", choice)
```

```
print("Reward :", reward)
```

```
print()
```

```
print("Total Reward :", total_reward)
```

Output



The screenshot displays an IDE with two windows. The left window, titled 'CO2-AT2-02.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-02.py (3.10.0)', contains the following Python code:

```
import random

# Study Materials
materials = ["Math", "Science", "AI", "Python"]

# Expected rewards for each material
rewards = {
    "Math": 6,
    "Science": 7,
    "AI": 8,
    "Python": 10
}

epsilon = 0.2
total_reward = 0

print("E-Learning Recommendation System\n")

for i in range(10):

    # Exploration or Exploitation
    if random.random() < epsilon:
        choice = random.choice(materials)
        strategy = "Exploration"
    else:
        choice = max(rewards, key=rewards.get)
        strategy = "Exploitation"

    reward = rewards[choice]
    total_reward += reward

    print("Recommendation", i + 1)
    print("Strategy :", strategy)
    print("Recommended Material :", choice)
    print("Reward :", reward)
    print()

print("Total Reward :", total_reward)
```

The right window, titled 'IDLE Shell 3.10.0', shows the output of the program:

```
Recommendation 1
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 2
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 3
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 4
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 5
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 6
Strategy : Exploration
Recommended Material : Python
Reward : 10

Recommendation 7
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 8
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 9
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Recommendation 10
Strategy : Exploitation
Recommended Material : Python
Reward : 10

Total Reward : 100
```

3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. (5 Marks)

States

- Current grid position / map coordinates
- Dirt level sensed at current location
- Battery level
- Distance to nearest obstacle

Actions

- Move forward/backward/left/right
- Activate suction / change cleaning mode
- Return to charging dock

Reward Function

- + reward for cleaning a dirty cell
- – penalty for collisions with obstacles
- – small penalty per time step (encourages efficiency)
- + reward for docking to recharge before battery depletes

Explanation

The policy is the mapping from each sensed state to the best action (e.g., 'if dirt detected ahead, move forward and clean; if obstacle detected, turn'). The value function estimates the total expected future reward from a given state, not just the immediate reward — so the agent learns to prefer paths that cover more uncleaned area overall, even if a nearby small reward looks tempting in the short term. As the agent explores the room repeatedly, value estimates are updated (e.g., via temporal-difference learning), and the policy derived from these estimates becomes increasingly efficient — covering the room faster with fewer repeated passes and fewer collisions.

Program

```
# Vacuum Cleaner Reinforcement Learning Simulation
```

```
grid = [  
    [0, 1, 0],  
    [1, 0, 1],  
    [0, 0, 1]  
]
```

```
reward = 0
```

```
cleaned = 0
```

```
moves = 0
```

```
print("=== Vacuum Cleaner Simulation ===\n")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        state = (i, j)
```

```
        action = "Move"
```

```
        print("Current State :", state)
```

```
        print("Action :", action)
```

```
moves += 1
```

```
if grid[i][j] == 1:
```

```
    print("Status : Dirt Detected")
```

```
    print("Action : Cleaning Dirt")
```

```
    grid[i][j] = 0
```

```
    cleaned += 1
```

```
    reward += 10
```

```
else:
```

```
    print("Status : No Dirt")
```

```
    print("Action : Continue Moving")
```

```
    reward -= 1
```

```
print("Current Reward :", reward)
```

```
print("-" * 30)
```

```
print("\n=== Cleaning Completed ===")
```

```
print("Total Cells Visited :", moves)
```

```
print("Total Dirt Cleaned :", cleaned)
```

```
print("Final Reward :", reward)
```

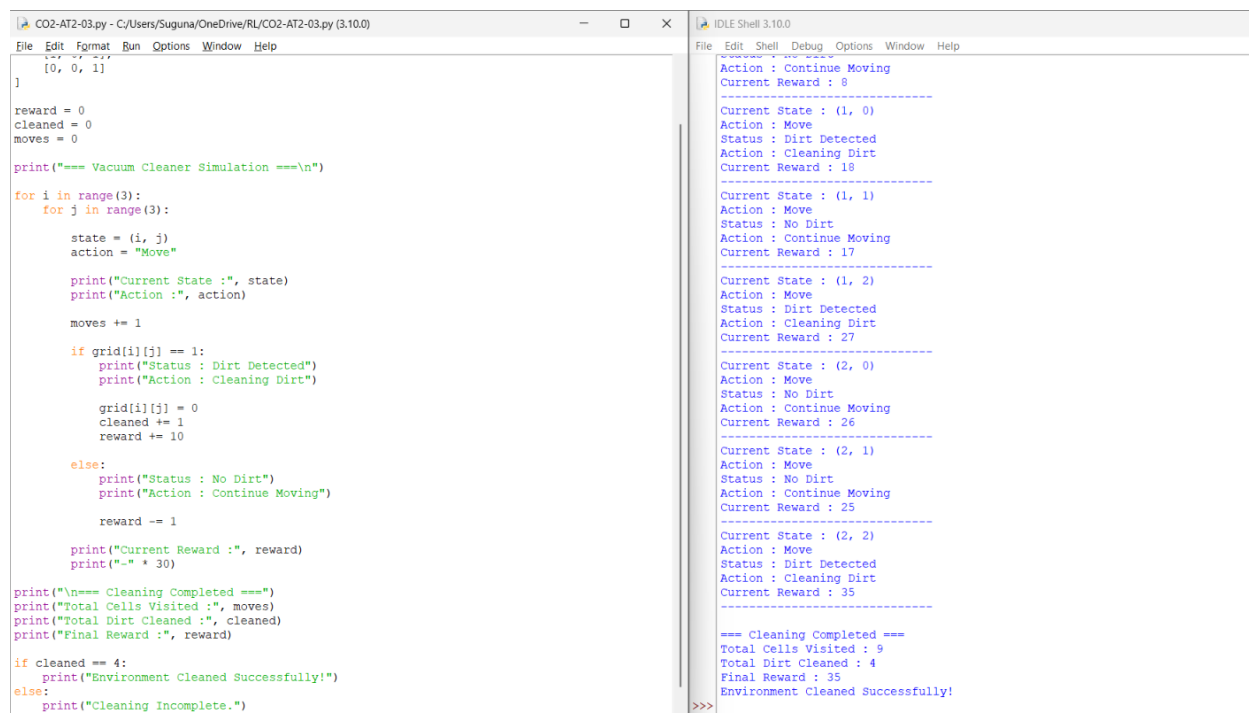

if cleaned == 4:

print("Environment Cleaned Successfully!")

else:

print("Cleaning Incomplete.")

Output



The screenshot displays a Python IDE with two windows. The left window, titled 'CO2-AT2-03.py', contains the following code:

```
[0, 0, 1]
]

reward = 0
cleaned = 0
moves = 0

print("=== Vacuum Cleaner Simulation ===\n")
for i in range(3):
    for j in range(3):
        state = (i, j)
        action = "Move"

        print("Current State :", state)
        print("Action :", action)

        moves += 1

        if grid[i][j] == 1:
            print("Status : Dirt Detected")
            print("Action : Cleaning Dirt")

            grid[i][j] = 0
            cleaned += 1
            reward += 10

        else:
            print("Status : No Dirt")
            print("Action : Continue Moving")

            reward -= 1

        print("Current Reward :", reward)
        print("-" * 30)

print("\n=== Cleaning Completed ===")
print("Total Cells Visited :", moves)
print("Total Dirt Cleaned :", cleaned)
print("Final Reward :", reward)

if cleaned == 4:
    print("Environment Cleaned Successfully!")
else:
    print("Cleaning Incomplete.")
```

The right window, titled 'IDLE Shell 3.10.0', shows the output of the program:

```
Action : Continue Moving
Current Reward : 8
-----
Current State : (1, 0)
Action : Move
Status : Dirt Detected
Action : Cleaning Dirt
Current Reward : 18
-----
Current State : (1, 1)
Action : Move
Status : No Dirt
Action : Continue Moving
Current Reward : 17
-----
Current State : (1, 2)
Action : Move
Status : Dirt Detected
Action : Cleaning Dirt
Current Reward : 27
-----
Current State : (2, 0)
Action : Move
Status : No Dirt
Action : Continue Moving
Current Reward : 26
-----
Current State : (2, 1)
Action : Move
Status : No Dirt
Action : Continue Moving
Current Reward : 25
-----
Current State : (2, 2)
Action : Move
Status : Dirt Detected
Action : Cleaning Dirt
Current Reward : 35
-----
=== Cleaning Completed ===
Total Cells Visited : 9
Total Dirt Cleaned : 4
Final Reward : 35
Environment Cleaned Successfully!
>>>
```

4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. (5 Marks)

Key Reward-Design Challenges

- Sparse and delayed rewards — real profit/loss is only known once a position is closed, not at the moment of each action
- High market volatility and noise make it hard to distinguish a genuinely good decision from a lucky short-term price move
- Non-stationary environment — market behavior shifts over time, so a reward function tuned to past data may no longer reflect good decisions
- Risk is not captured by raw profit alone — a reward based only on price change ignores exposure/drawdown risk

Explanation

If the reward is defined purely as immediate price change after each action, the agent tends to over-trade on short-term noise, mistaking random fluctuations for exploitable patterns, and ignores portfolio risk. This can produce a policy that looks profitable in backtesting but takes oversized risky positions in practice, leading to large drawdowns or account 'blow-ups' when volatility spikes. Good reward design must incorporate risk-adjusted returns (e.g., penalizing large drawdowns or position sizes) rather than raw profit alone, to produce a stable, generalizable trading policy.

Program

```
import random
```

```
# Possible actions
```

```
actions = ["Buy", "Sell", "Hold"]
```

```
price = 100
```

```
balance = 1000
```

```
total_reward = 0
```

```
print("=== Cryptocurrency Trading Simulation ===\n")
```

```
for day in range(1, 6):
```

```
    action = random.choice(actions)
```

```
    change = random.randint(-10, 10)
```

```
    old_price = price
```

```
    price += change
```

```
    if action == "Buy":
```

```
        reward = max(0, change)
```

```
        balance -= price
```

```
    elif action == "Sell":
```

```
        reward = max(0, -change)
```

```
        balance += price
```

```
    else: # Hold
```

```
        reward = 2
```

```
total_reward += reward
```

```
print("Day :", day)
```

```
print("Previous Price :", old_price)
```

```
print("Current Price :", price)
```

```
print("Action Taken :", action)
```

```
print("Reward :", reward)
```

```
print("Balance :", balance)
```

```
print("-" * 35)
```

```
print("\n=== Trading Summary ===")
```

```
print("Final Coin Price :", price)
```

```
print("Final Balance :", balance)
```

```
print("Total Reward :", total_reward)
```

```
if total_reward > 20:
```

```
    print("Trading Performance : Good")
```

```
else:
```

```
    print("Trading Performance : Needs Improvement")
```

Output

```
CO2-AT2-04.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-04.py (3.10.0)
File Edit Format Run Options Window Help
actions = ["Buy", "Sell", "Hold"]

price = 100
balance = 1000
total_reward = 0

print("=== Cryptocurrency Trading Simulation ===\n")

for day in range(1, 6):

    action = random.choice(actions)
    change = random.randint(-10, 10)

    old_price = price
    price += change

    if action == "Buy":
        reward = max(0, change)
        balance -= price

    elif action == "Sell":
        reward = max(0, -change)
        balance += price

    else: # Hold
        reward = 2

    total_reward += reward

    print("Day :", day)
    print("Previous Price :", old_price)
    print("Current Price :", price)
    print("Action Taken :", action)
    print("Reward :", reward)
    print("Balance :", balance)
    print("-" * 35)

print("\n=== Trading Summary ===")
print("Final Coin Price :", price)
print("Final Balance :", balance)
print("Total Reward :", total_reward)

if total_reward > 20:
    print("Trading Performance : Good")
else:
    print("Trading Performance : Needs Improvement")

===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-04.py =====
=== Cryptocurrency Trading Simulation ===

Day : 1
Previous Price : 100
Current Price : 97
Action Taken : Hold
Reward : 2
Balance : 1000
-----
Day : 2
Previous Price : 97
Current Price : 94
Action Taken : Hold
Reward : 2
Balance : 1000
-----
Day : 3
Previous Price : 94
Current Price : 104
Action Taken : Sell
Reward : 0
Balance : 1104
-----
Day : 4
Previous Price : 104
Current Price : 99
Action Taken : Hold
Reward : 2
Balance : 1104
-----
Day : 5
Previous Price : 99
Current Price : 106
Action Taken : Hold
Reward : 2
Balance : 1104
-----

=== Trading Summary ===
Final Coin Price : 106
Final Balance : 1104
Total Reward : 8
Trading Performance : Needs Improvement
>>>
```

5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. (5 Marks)

State Space

- Current electricity demand per zone/region
- Available generation capacity (conventional + renewable)
- Energy storage / battery charge levels
- Time of day and weather forecast (affects solar/wind supply)

Action Space

- Increase/decrease supply allocation to a zone
- Charge or discharge storage/battery units
- Activate or deactivate backup generation
- Apply demand-response signals (e.g., dynamic pricing to shift load)

Reward Mechanism

- + reward for meeting demand reliably without shortages
- + reward for minimizing wasted/over-generated energy
- – large penalty for blackouts or load-shedding events
- – penalty for sharp peak-load spikes

Explanation

This reward mechanism directly balances the grid's two competing goals — reliability (always meeting demand) and efficiency (not wasting generated power) — while strongly discouraging blackouts, which guides the learned policy toward smooth, proactive load balancing rather than reactive emergency responses.

Program

```
import random
```

```
# Electricity demand in different zones
```

```
demand = [60, 80, 40, 70, 90]
```

```
total_reward = 0
```

```
print("=== Smart Grid Electricity Distribution ===\n")
```

```
for zone in range(len(demand)):
```

```
    current_demand = demand[zone]
```

```
    supply = random.randint(50, 100)
```

```
    print("Zone :", zone + 1)
```

```
    print("Demand :", current_demand)
```

```
    print("Supply :", supply)
```

```
    if supply >= current_demand:
```

```
        action = "Supply Electricity"
```

```
        status = "Demand Met"
```

```
        reward = 10
```

```
    else:
```

```
        action = "Increase Power Generation"
```

```
        status = "Power Shortage"
```

```
        reward = -10
```

```
total_reward += reward
```

```
print("Action :", action)
```

```
print("Status :", status)
```

```
print("Reward :", reward)
```

```
print("-" * 35)
```

```
print("\n=== Smart Grid Summary ===")
```

```
print("Total Reward :", total_reward)
```

```
if total_reward > 0:
```

```
    print("Grid Performance : Efficient")
```

```
else:
```

```
print("Grid Performance : Needs Improvement")
```


Output

```
CO2-AT2-05.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-05.py (3.10.0)
File Edit Format Run Options Window Help
import random
# Electricity demand in different zones
demand = [60, 80, 40, 70, 90]
total_reward = 0
print("=== Smart Grid Electricity Distribution ===\n")
for zone in range(len(demand)):
    current_demand = demand[zone]
    supply = random.randint(50, 100)
    print("Zone :", zone + 1)
    print("Demand :", current_demand)
    print("Supply :", supply)
    if supply >= current_demand:
        action = "Supply Electricity"
        status = "Demand Met"
        reward = 10
    else:
        action = "Increase Power Generation"
        status = "Power Shortage"
        reward = -10
    total_reward += reward
    print("Action :", action)
    print("Status :", status)
    print("Reward :", reward)
    print("-" * 35)
print("\n=== Smart Grid Summary ===")
print("Total Reward :", total_reward)
if total_reward > 0:
    print("Grid Performance : Efficient")
else:
    print("Grid Performance : Needs Improvement")

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
=====
Total Reward : 8
Trading Performance : Needs Improvement
>>>
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-05.py =====
=== Smart Grid Electricity Distribution ===
Zone : 1
Demand : 60
Supply : 96
Action : Supply Electricity
Status : Demand Met
Reward : 10
-----
Zone : 2
Demand : 80
Supply : 59
Action : Increase Power Generation
Status : Power Shortage
Reward : -10
-----
Zone : 3
Demand : 40
Supply : 65
Action : Supply Electricity
Status : Demand Met
Reward : 10
-----
Zone : 4
Demand : 70
Supply : 82
Action : Supply Electricity
Status : Demand Met
Reward : 10
-----
Zone : 5
Demand : 90
Supply : 80
Action : Increase Power Generation
Status : Power Shortage
Reward : -10
-----
=== Smart Grid Summary ===
Total Reward : 10
Grid Performance : Efficient
```

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. (5 Marks)

States

- Patient vital signs (heart rate, blood pressure, blood glucose, etc.)
- Current medication dosage level
- Time since last dose
- Patient-specific history / response pattern

Actions

- Increase dosage
- Decrease dosage
- Maintain current dosage
- Alert a clinician for manual review

Rewards

- + reward for vitals remaining within the safe/target range
- – large penalty for an adverse reaction or vitals leaving the safe range
- – small penalty for unnecessary/frequent dosage changes (encourages stability)

Explanation

Because patients respond differently to the same dosage, a fixed one-size-fits-all schedule is often suboptimal. Through repeated interaction, the RL agent learns each patient's individual dose-response pattern and gradually adjusts dosing to keep vitals stable faster and with fewer adverse events than a static protocol — while the strong penalty on unsafe vitals keeps the learned policy conservative and safety-first.

Program

```
import random
```

```
# Initial dosage
```

```
dosage = 50
```

```
total_reward = 0
```

```
print("=== Healthcare Monitoring System ===\n")
```

```
for day in range(1, 6):
```

```
    vital = random.randint(60, 100)
```

```
    print("Day :", day)
```

```
    print("Patient Vital Sign :", vital)
```

```
    if vital < 70:
```

```
        action = "Increase Dosage"
```

```
        dosage += 5
```

```
        reward = 10
```

```
        status = "Patient Needs More Medication"
```

```
    elif vital > 90:
```

```
        action = "Decrease Dosage"
```

```
        dosage -= 5
```

```
        reward = 8
```

```
        status = "Patient Stable"
```

```
    else:
```

```
action = "Maintain Dosage"
```

```
reward = 15
```

```
status = "Vital Signs Normal"
```

```
total_reward += reward
```

```
print("Action :", action)
```

```
print("Current Dosage :", dosage)
```

```
print("Patient Status :", status)
```

```
print("Reward :", reward)
```

```
print("-" * 35)
```

```
print("\n=== Monitoring Summary ===")
```

```
print("Final Dosage :", dosage)
```

```
print("Total Reward :", total_reward)
```

```
if total_reward >= 60:
```

```
    print("Treatment Result : Successful")
```

```
else:
```

```
    print("Treatment Result : Needs Improvement")
```

Output

```
CO2-AT2-06.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-06.py (3.10.0)
File Edit Format Run Options Window Help

# Initial dosage
dosage = 50
total_reward = 0

print("=== Healthcare Monitoring System ===\n")

for day in range(1, 6):

    vital = random.randint(60, 100)

    print("Day :", day)
    print("Patient Vital Sign :", vital)

    if vital < 70:
        action = "Increase Dosage"
        dosage += 5
        reward = 10
        status = "Patient Needs More Medication"

    elif vital > 90:
        action = "Decrease Dosage"
        dosage -= 5
        reward = 8
        status = "Patient Stable"

    else:
        action = "Maintain Dosage"
        reward = 15
        status = "Vital Signs Normal"

    total_reward += reward

    print("Action :", action)
    print("Current Dosage :", dosage)
    print("Patient Status :", status)
    print("Reward :", reward)
    print("-" * 35)

print("\n=== Monitoring Summary ===")
print("Final Dosage :", dosage)
print("Total Reward :", total_reward)

if total_reward >= 60:
    print("Treatment Result : Successful")
else:
    print("Treatment Result : Needs Improvement")

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help

Grid Performance : Efficient

>>>
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-06.py =====
=== Healthcare Monitoring System ===

Day : 1
Patient Vital Sign : 65
Action : Increase Dosage
Current Dosage : 55
Patient Status : Patient Needs More Medication
Reward : 10
-----
Day : 2
Patient Vital Sign : 80
Action : Maintain Dosage
Current Dosage : 55
Patient Status : Vital Signs Normal
Reward : 15
-----
Day : 3
Patient Vital Sign : 89
Action : Maintain Dosage
Current Dosage : 55
Patient Status : Vital Signs Normal
Reward : 15
-----
Day : 4
Patient Vital Sign : 73
Action : Maintain Dosage
Current Dosage : 55
Patient Status : Vital Signs Normal
Reward : 15
-----
Day : 5
Patient Vital Sign : 72
Action : Maintain Dosage
Current Dosage : 55
Patient Status : Vital Signs Normal
Reward : 15
-----

=== Monitoring Summary ===
Final Dosage : 55
Total Reward : 70
Treatment Result : Successful

>>>
```

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. (5 Marks)

Exploitation

- Reuse routes that have previously proven fast and safe (known good paths)

Exploration

- Occasionally try alternative routes to discover shortcuts or adapt to new obstacles, weather, or no-fly zones

Explanation

If the drone only exploits known routes, it cannot adapt when conditions change (e.g., new construction blocking a previously optimal path), leading to persistently suboptimal or even unsafe navigation. If it explores too much, it wastes battery and delivery time testing unproven paths. A decaying exploration strategy — trying more alternatives early on, then increasingly favoring the best-known routes as confidence grows — lets the system discover efficient paths initially while still periodically re-checking for better options as the environment changes, which is essential for continuously optimized delivery routing.

Program

```
import random
```

```
routes = {  
    "Route A": 8,  
    "Route B": 6,  
    "Route C": 10  
}
```

```
epsilon = 0.3
```

```
total_reward = 0
```

```
print("Drone Delivery Navigation\n")

for i in range(10):

    if random.random() < epsilon:

        route = random.choice(list(routes.keys()))

        strategy = "Exploration"

    else:

        route = max(routes, key=routes.get)

        strategy = "Exploitation"

    reward = routes[route]

    total_reward += reward

    print("Trip", i + 1)

    print("Strategy :", strategy)

    print("Selected Route :", route)

    print("Reward :", reward)

    print()

    print("Total Reward =", total_reward)
```

Output

```
CO2-AT2-07.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-07.py (3.10.0)
File Edit Format Run Options Window Help

import random

routes = {
    "Route A": 8,
    "Route B": 6,
    "Route C": 10
}

epsilon = 0.3
total_reward = 0

print("Drone Delivery Navigation\n")

for i in range(10):
    if random.random() < epsilon:
        route = random.choice(list(routes.keys()))
        strategy = "Exploration"
    else:
        route = max(routes, key=routes.get)
        strategy = "Exploitation"

    reward = routes[route]
    total_reward += reward

    print("Trip", i + 1)
    print("Strategy :", strategy)
    print("Selected Route :", route)
    print("Reward :", reward)
    print()

print("Total Reward =", total_reward)
|

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help

---
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Trip 3
Strategy : Exploration
Selected Route : Route B
Reward : 6

Trip 4
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Trip 5
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Trip 6
Strategy : Exploration
Selected Route : Route A
Reward : 8

Trip 7
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Trip 8
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Trip 9
Strategy : Exploration
Selected Route : Route B
Reward : 6

Trip 10
Strategy : Exploitation
Selected Route : Route C
Reward : 10

Total Reward = 90
```


8. Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. (5 Marks)

States

- Current assembly stage / step
- Part positions and orientation
- Robot arm position
- Sensor readings for misalignment/error

Actions

- Move arm to a target position
- Grip / release a part
- Insert or place a part
- Adjust applied force
- Verify assembly correctness

Reward Function

- + reward for each correctly completed assembly step
- – penalty for misalignment or assembly errors
- – small penalty per time step (encourages speed)
- + bonus for completing the full assembly within a target time

Explanation

The policy maps the robot's current sensed state to the best next action for the assembly task. The value function estimates the total expected future reward from a given state — not just the immediate step's reward — so the robot learns to favor actions that set up smoother, faster downstream steps (e.g., positioning a part in a way that eases the next insertion), rather than greedily optimizing only the current action. As value estimates are refined through repeated assembly cycles, the resulting policy becomes progressively faster and more error-free.

Program

steps = ["Pick Part", "Move Arm", "Place Part", "Fix Screw", "Inspection"]

```
total_reward = 0
```

```
print("Manufacturing Robot Assembly\n")
```

```
for step in steps:
```

```
    print("Current Step :", step)
```

```
        success = True
```

```
            if success:
```

```
                reward = 10
```

```
                print("Status : Completed")
```

```
            else:
```

```
                reward = -5
```

```
                print("Status : Error")
```

```
        total_reward += reward
```

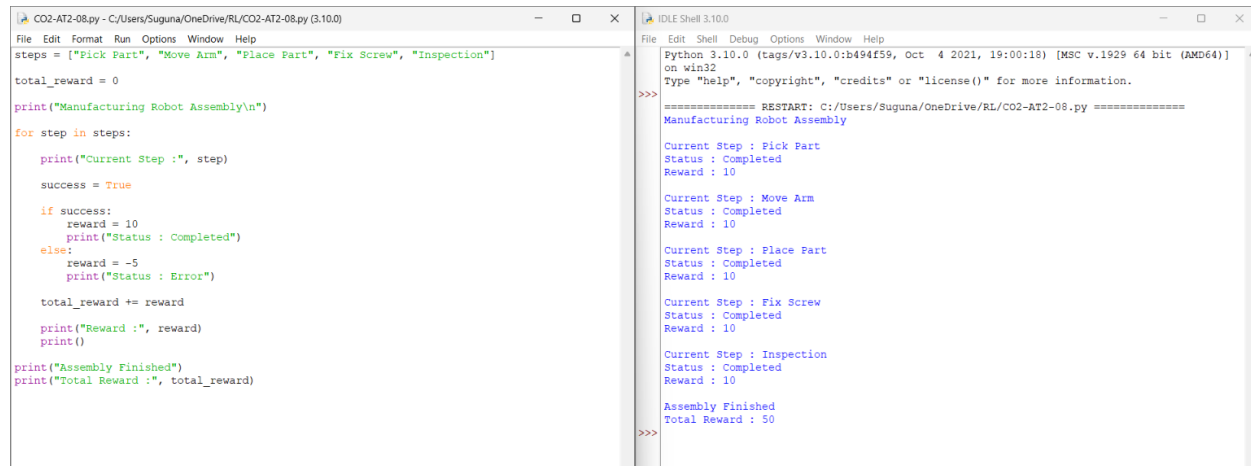
```
    print("Reward :", reward)
```

```
    print()
```

```
print("Assembly Finished")
```

```
    print("Total Reward :", total_reward)
```

Output



```
CO2-AT2-08.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-08.py (3.10.0)
File Edit Format Run Options Window Help
steps = ["Pick Part", "Move Arm", "Place Part", "Fix Screw", "Inspection"]
total_reward = 0
print("Manufacturing Robot Assembly\n")
for step in steps:
    print("Current Step :", step)
    success = True
    if success:
        reward = 10
        print("Status : Completed")
    else:
        reward = -5
        print("Status : Error")
    total_reward += reward
    print("Reward :", reward)
    print()
print("Assembly Finished")
print("Total Reward :", total_reward)
```

```
IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-08.py =====
Manufacturing Robot Assembly

Current Step : Pick Part
Status : Completed
Reward : 10

Current Step : Move Arm
Status : Completed
Reward : 10

Current Step : Place Part
Status : Completed
Reward : 10

Current Step : Fix Screw
Status : Completed
Reward : 10

Current Step : Inspection
Status : Completed
Reward : 10

Assembly Finished
Total Reward : 50
>>>
```

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. (5 Marks)

Key Reward-Design Challenges

- Reward sparsity — clicks are rare relative to the number of ad impressions shown
- Delayed/attribution issues — a conversion (e.g., purchase) may happen long after the click, making it hard to credit the right ad/action
- Reward hacking risk — optimizing purely for clicks can favor sensational or misleading ('clickbait') ads
- Feedback-loop bias — the system only observes outcomes for ads it chose to show, biasing future learning

Explanation

If the reward function is based purely on click-through rate, the agent can learn to favor attention-grabbing but low-quality or misleading ads, which may raise short-term CTR while damaging user trust, brand reputation, and advertiser satisfaction in the long run. Reward sparsity also slows convergence, since useful learning signal (a click) is infrequent. This is typically mitigated with reward shaping — combining CTR with auxiliary signals like dwell time, conversion rate, or user satisfaction scores — to produce a policy that optimizes genuine engagement rather than exploiting the click-reward signal alone.

Program

```
import random
```

```
ads = ["Ad A", "Ad B", "Ad C"]
```

```
total_reward = 0
```

```
print("Online Advertising System\n")
```

```
for i in range(10):
```

```
ad = random.choice(ads)
```

```
clicked = random.choice([True, False])
```

```
if clicked:
```

```
    reward = 10
```

```
else:
```

```
    reward = -2
```

```
total_reward += reward
```

```
print("Advertisement :", ad)
```

```
print("Clicked :", clicked)
```

```
print("Reward :", reward)
```

```
print()
```

```
print("Total Reward :", total_reward)
```

Output

```
CO2-AT2-09.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-09.py (3.10.0)
File Edit Format Run Options Window Help
import random
ads = ["Ad A", "Ad B", "Ad C"]
total_reward = 0
print("Online Advertising System\n")
for i in range(10):
    ad = random.choice(ads)
    clicked = random.choice([True, False])
    if clicked:
        reward = 10
    else:
        reward = -2
    total_reward += reward
    print("Advertisement :", ad)
    print("Clicked :", clicked)
    print("Reward :", reward)
    print()
print("Total Reward :", total_reward)

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
>>> ===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-09.py =====
Online Advertising System

Advertisement : Ad B
Clicked : True
Reward : 10

Advertisement : Ad B
Clicked : False
Reward : -2

Advertisement : Ad A
Clicked : True
Reward : 10

Advertisement : Ad A
Clicked : False
Reward : -2

Advertisement : Ad C
Clicked : True
Reward : 10

Advertisement : Ad B
Clicked : True
Reward : 10

Advertisement : Ad C
Clicked : True
Reward : 10

Advertisement : Ad A
Clicked : False
Reward : -2

Advertisement : Ad B
Clicked : False
Reward : -2

Advertisement : Ad C
Clicked : False
Reward : -2

Total Reward : 40
>>>
```

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. (5 Marks)

States

- Current soil moisture level
- Weather forecast (rain probability, temperature)
- Crop growth stage
- Water reservoir/tank level

Actions

- Turn irrigation on/off for a zone
- Adjust water flow rate / duration

Reward Function

- + reward for maintaining soil moisture within the optimal range for the crop stage
- – penalty for water wastage/over-irrigation
- – penalty for under-watering (risk to crop health)
- + small bonus for water conservation when moisture is already adequate

Explanation

This reward function directly balances two competing goals — crop health (needs sufficient water) and resource conservation (avoiding waste) — so the learned policy irrigates only when needed and by the right amount, adapting to weather forecasts (e.g., skipping irrigation if rain is imminent) rather than following a fixed schedule.

Program

```
import random
```

```
print("Smart Irrigation System\n")
```

```
total_reward = 0
```

```
for day in range(1, 8):
```

```
moisture = random.randint(20, 80)
```

```
if moisture < 40:
```

```
    action = "Irrigation ON"
```

```
    reward = 10
```

```
elif moisture <= 60:
```

```
    action = "Monitor Soil"
```

```
    reward = 8
```

```
else:
```

```
    action = "Irrigation OFF"
```

```
    reward = 5
```

```
total_reward += reward
```

```
print("Day :", day)
```

```
print("Soil Moisture :", moisture)
```

```
print("Action :", action)
```

```
print("Reward :", reward)
```

```
print()
```

```
print("Total Reward :", total_reward)
```

Output

```
CO2-AT2-10.py - C:/Users/Suguna/OneDrive/RL/CO2-AT2-10.py (3.10.0)
File Edit Format Run Options Window Help
import random

print("Smart Irrigation System\n")

total_reward = 0

for day in range(1, 8):

    moisture = random.randint(20, 80)

    if moisture < 40:
        action = "Irrigation ON"
        reward = 10
    elif moisture <= 60:
        action = "Monitor Soil"
        reward = 8
    else:
        action = "Irrigation OFF"
        reward = 5

    total_reward += reward

    print("Day :", day)
    print("Soil Moisture :", moisture)
    print("Action :", action)
    print("Reward :", reward)
    print()

print("Total Reward :", total_reward)
```

```
IDLE Shell 3.10.0
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Suguna/OneDrive/RL/CO2-AT2-10.py =====
Smart Irrigation System

Day : 1
Soil Moisture : 46
Action : Monitor Soil
Reward : 8

Day : 2
Soil Moisture : 58
Action : Monitor Soil
Reward : 8

Day : 3
Soil Moisture : 28
Action : Irrigation ON
Reward : 10

Day : 4
Soil Moisture : 79
Action : Irrigation OFF
Reward : 5

Day : 5
Soil Moisture : 59
Action : Monitor Soil
Reward : 8

Day : 6
Soil Moisture : 42
Action : Monitor Soil
Reward : 8

Day : 7
Soil Moisture : 53
Action : Monitor Soil
Reward : 8

Total Reward : 55
>>>
```

Ln:30 Col:0

Ln:43 C